



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE THREE: JAVA FUNDAMENTALS

UNIT ONE: JAVA BASICS - DATA TYPES, VARIABLES AND OPERATORS IN JAVA PART I

JAVA BASICS - DATA TYPES, VARIABLES AND OPERATORS IN JAVA PART I

Learning Objectives

By the end of this unit, you will have been introduced to:

- **data types in Java and their associated memory sizes.**
- **the correct syntax for declaring and using variables.**
- **operators like arithmetic, relational, and logical operators.**
- **various forms of control structures used in Java.**
- **the process of writing, compiling, and running Java programs.**
- **techniques for handling user input and output in Java**
- **the overall structure a Java program**

Learning Outcomes

At the end of this unit, you should be able to:

- **explain Java data types and their memory sizes.**
- **declare and use variables with correct naming conventions.**
- **identify various operators used in Java including arithmetic, relational, etc.**
- **identify the various forms of control structures**
- **write and compile basic Java programs.**
- **handle user input and display out.**
- **understand the structure of a Java program.**

Introduction

- **To develop efficient and reliable Java applications, mastering core programming concepts is essential. This unit provides a comprehensive introduction to Java fundamentals, including data types, variables, and operators.**
- **You will be introduced to the types of control structures for program flow management and learn the complete process of writing, compiling, and executing simple Java programs.**
- **The unit also covers practical input/output operations, teaching various methods for user interaction and data display.**

Basic Concepts in Programming

Programming and writing logical program begin with mastering basic concepts that serve as building blocks for all code. The core elements include:

- **variables,**
- **data types,**
- **operators, and**
- **control structures.**

Variables

A variable is a container that stores data. Variables store values such as numbers, characters, text, or logical values. A variable in Java must be declared before it is used. You declare a variable by specifying its data type followed by the variable name. For example:

int number;

where int is the data type and number is the name of the variable.

Rules for Naming Variables

- 1. Variable names must start with a letter, underscore (_) or dollar sign (\$). For instance _myName, \$fileName, person, etc. are valid variable names.**
- 2. Variable name cannot start with a digit. 1variable, 123LondonStreet, 2name, etc. are invalid variable names.**
- 3. A variable name can include letters (A to Z, a to z), digits (0 to 9), underscore (_), and dollar sign (\$). name2, variable\$25, input22, etc are all valid variable names.**

Rules for Naming Variables (2)

- 4. Variable names are case-sensitive, meaning age, AGE, and Age are different.**
- 5. Variable names cannot contain spaces, or special characters like exclamation (!), at-sign (@), hyphen (-), etc. my-variable and my@name are invalid.**
- 6. Variable names cannot be keyword a. int, class, float, if, public, else, private, etc. are invalid variable names.**

Data Types

Data types are used to specify the type of data (value) a variable can hold (store). The type of data a variable can store determines the operations that can be performed on that variable. For instance, consider the code snippet below:

```
public class Main {  
    public static void main(String[] args) {  
        int age = 25;  
        // 'age' is the variable 'int' is the  
        // data type and 25 is the value  
  
        // Printing the value of the variable 'age'  
        System.out.println("The age is: " + age);  
    }  
}
```

Primitive vs Non Primitive Data Types

- **Primitive data types are the most basic type of data in Java. They represent single values. Examples include byte, short, int (integer), float, double, char (character), and boolean.**
- **Non primitive data types are also known as reference types because they refer to objects in memory. These data types can store objects and are user-defined. Examples include: Classes, Interfaces, Arrays and String.**

Primitive Data Types

Data type	Size	Default Value	Description
byte	1 byte	0	Stores 8-bit signed integers. Range is -128 to 127
short	2 bytes	0	Stores 16-bit signed integers. Range is -32,768 to 32,767
int	4 bytes	0	Stores 32-bit signed integers (whole numbers) Range is - 2^{31} to $2^{31} - 1$ (that is, -2,147,483,648 to 2,147,483,648)
long	8 bytes	0L	Stores 64-bit signed integers (whole numbers). Range is 2^{63} to $2^{63}-1$
float	4 bytes	0.0f	Stores 32-bit floating-point numbers (decimal numbers). Precision is up to 7 digits.
double	8 bytes	0.0d	Stores 64-bit floating-point numbers (decimal numbers). Precision is up to 15 digits.
char	2 bytes	'\u0000'	Stores a single 16-bit Unicode character. It is enclosed in single quotes ('A', '1').
boolean	1 bite	false	Stores true or false

Non Primitive Data Types

- 1. Classes:** A class is a template used to create objects. It defines the properties (variables) and behaviour (methods) that an object can have.
- 2. Interfaces:** An interface is an abstract type that defines a set methods that a class must implement.
- 3. Arrays:** An array is a data structure that can hold multiple values of the same type.
- 4. Strings:** String is an object and is used to represent a sequence of characters.

Operators

Operators are symbols that perform operations on variables, constants or values. The following are types of operators:

- 1. Arithmetic Operators**
- 2. Relational (Comparison) Operators**
- 3. Logical Operators**
- 4. Assignment Operators**
- 5. Unary Operators**
- 6. Ternary Operator**
- 7. Instanceof Operator**
- 8. Bitwise Operators**

Operators

- 1. Arithmetic Operators:** They are used to perform basic mathematics operations such as addition (+), subtraction (-), division (/), multiplication (*) and modulus (%).
- 2. Relational (Comparison) Operators:** They are used to compare two values. These operators return a boolean value (true or false) Examples include equal to (==), not equal to (!=), greater than (>), greater than or equal to (>=), less than (<), and less than or equal to (<=).
- 3. Logical Operators:** Logical operators are used to perform logical operations. Examples include Logical AND (&&), Logical OR (||) and Logical NOT (!)

Operators

- 4. Assignment Operators:** Assignment operators are used to assign values to variables. These operators can also perform operations and assign the result back to the variable. Examples include simple assignment (`=`), addition assignment (`+=`), subtraction assignment (`-=`), multiplication assignment (`*=`) and division assignment (`/=`).
- 5. Unary Operators:** Unary operators are used with a single operand to perform operation such as increasing, decreasing or negating. Examples include bitwise complement (`~a`), unary plus (`+a`), unary minus (`-a`), increment (`a++`, `++a`), decrement (`a--`, `--a`), etc.

Operators

6. **Ternary Operator (?:):** The ternary operator is a shorthand for if-else statements. It takes three operands and returns one of two values depending on the condition. The syntax is

condition ? value1 : value2

6. **Instanceof Operator:** The instanceof operator is used to check whether an object is an instance of a specific class or implements an interface. The Syntax follows the format `objectName instanceof Class`
7. **Bitwise Operators:** Bitwise operators perform operations on the binary representations of integers.

Control Structure

Control structures dictate the flow of execution in a program. By default, codes are executed linearly from top to bottom. There are three main types of control structures:

- 1. Selection Control (Decision Making): Executes different codes based on conditions. Examples include if, if-else, switch statements.**
- 2. Iteration Control (Loops): Repeats a block of code. Example includes for, while and do while loops.**
- 3. Branching Statements: Alters the normal flow using break, continue and return.**

Steps to Write and Run a Java Program

In Java programming language, all source codes are written in plain text file saved with a .java extension. These sources codes are compiled into bytecodes (the machine language of the Java Virtual Machine) by the compiler. The bytecodes are saved in a file with a .class extension. The steps to writing and running a Java program include:

- **write the source code (.java),**
- **compile the source code and**
- **run the compiled code.**

The Steps: Writing the Code

The steps include:

- i. Open a text editor (e.g., Notepad, VS Code, IntelliJ IDEA, Eclipse).**
- ii. Type the Java program and save it with same name as the Class and use the .java extension e.g. Hello.java.**

The Steps: Compiling the Code

Java programs need to be compiled before execution.

- i. Open the Command Prompt and navigate to the directory where the file is saved.**
- ii. Compile the code with the command**

javac Hello.java

If there are no syntax errors, a compiled file with a .class is created in the directory.

The Steps: Running the Program

After successful compilation, run the compiled code using the command:

java Hello

Structure of a Simple Java Program

A basic Java program consists of the following components:

- 1. Class Declaration: Every Java code must have at least one class.**
- 2. Main Method (main): The entry point of any Java application.**
- 3. Statements: Instructions to be executed by the compiler.**
- 4. Comments (Optional): None executable statements.**

A simple Java program called FirstProgram is shown below:

Explanation of the Code

```
1  public class FirstProgram {  
2      public static void main (String[] args) {  
3          // Displays Hello Java  
4          System.out.println("Hello Java!");  
5      }  
6  }
```

1. **Class Declaration:** Line 1 defines the class with an access modifier of public and a class name of FirstProgram. By convention, class names start with an uppercase to distinguish them from variables/methods.

public class FirstProgram {

Explanation of the Code (2)

2. **The main Method:** Line 2 defines the main method (the entry point). **static** means that the method belongs to the class not an instance. **void** means that the method does not return any value, while **Strings[] args** indicates that the program accepts command-line arguments.

public static void main (String[] args) {

3. **Comment:** Line 3 is a comment that explain more about the line of code before it. Comments are not executed by the compiler.
4. **Printing Output:** Line 4 prints the string “Hello Java!” on the console.

Java Input

Input handling lets users provide data to an application from the keyboard, files, or networks. Common ways include:

- 1. Scanner Class (Most Common)**
- 2. BufferedReader and InputStreamReader**
- 3. Console (Secure for Passwords)**

Java Input - Using Scanner Class

The Scanner class is part of java.util package. It can read various types of inputs like strings, integers, floats, etc.

```
1  import java.util.Scanner;
2  public class ScannerInput {
3      public static void main(String[] args) {
4          Scanner input = new Scanner(System.in);
5
6          System.out.print("Enter your name: ");
7          String name = input.nextLine();
8          System.out.print("Welcome " + name);
9
10         input.close();
11     }
12 }
```

When the code above run, it will display,

Enter your name: John Doe

Welcome John Doe

Java Input - Using BufferedReader and InputStreamReader

```
import java.io.*;

public class BufferedReaderInput {
    public static void main(String[] args) throws IOException {
        BufferedReader br =
            new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Enter an integer: ");
        int n = Integer.parseInt(br.readLine());

        System.out.println("You entered: " + n);
        br.close();
    }
}
```

This program uses BufferedReader to read an integer input, converts it from String to int and prints the result.

Java Input - Using Console

```
import java.io.Console;

public class ConsoleExample {
    public static void main(String[] args) {
        Console console = System.console();

        if (console == null) {
            System.err.println("Console not available!");
            return;
        }

        String username = console.readLine("Enter username: ");
        char[] passwordChars = console.readPassword("Enter password: ");
        String password = new String(passwordChars);

        System.out.println("Username: " + username);

        // Just for demo (don't do this in real apps!)
        System.out.println("Password: " + password);
    }
}
```

Java Output

Java provides several ways to handle output, with the most common method being through the `System.out` stream. Here is a breakdown of different methods and concepts related to Java output:

- 1. Print with newline (`System.out.println()`)**
- 2. Print without newline (`System.out.print()`)**
- 3. Formatted Output (`System.out.printf()`)**

Java Output - Print with newline (System.out.println())

The println() method prints the specified text or value to the console and then moves the cursor to the next line.

```
public class Println {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
        // Prints the text and moves to a new line  
    }  
}
```

The output of the code is:

Hello, World

Java Output - Print without newline (System.out.print())

Unlike println(), print does not move the cursor to a new line after printing.

```
public class Print {  
    public static void main(String[] args) {  
        System.out.print("Hello, ");  
        System.out.print("Java!");  
    }  
}
```

The above code will output “**Hello Java**” on the same line.

Java Output - Formatted Output (System.out.printf())

printf() allows printing of variables with specific formatting (like setting the decimal places, padding numbers, etc.).

```
public class PrintFormatOne {  
    public static void main(String[] args) {  
        double pi = 3.14159;  
        System.out.printf("The value of Pi is %.2f", pi);  
        // Prints pi with two decimal places  
    }  
}
```

printf works with format specifiers. In the code, the format specifier is “%.2f”

Format Specifiers and Examples

Format	Description	Example
%s	String	"John"
%d	Integer	100
%f	Floating-point number	3.14159
%.2f	2 decimal places	3.14
%c	Character	'A'
%x or X%	Hexadecimal (lowercase or uppercase)	ff or FF
%e	Scientific notation (lowercase)	1.234568e+06
%E	Scientific notation (uppercase)	1.234568E+06
%05d	Padding with zeros	00005
 %+d or %-d	Forces a sign (+ or -)	+10 or -10
%-5d	Left justified e.g. " %-5d "	10
%5d	Minimum number of character	5

In-Text Questions (ITQs)

- 1. What is the purpose of the main method in a Java program?**
- 2. What are the differences between `System.out.print()`, `System.out.println()`, and `System.out.printf()`?**

In-Text Answers (ITAs)

- 1. The main method serves as the entry point for program execution. It is where the Java Virtual Machine (JVM) begins running the program. Its signature is public static void main(String[] args).**
- 2. The differences are:**
 - a. print() outputs text without moving to a new line.**
 - b. println() outputs text and moves the cursor to the next line.**
 - c. printf() allows formatted output using format specifiers (e.g., %d, %s, %.2f) for controlling the display of variables with precision, alignment, and padding.**

Conclusion

- **Mastering these Java fundamentals equips you with the basic tools needed to develop reliable and efficient Java applications.**
- **Understanding variables, data types, operators, and control structures forms the backbone of programming logic. With the ability to write, compile, and run Java code, and effectively manage user input and output, you are well-prepared to tackle more advanced programming challenges in Java.**

Summary

- **This unit introduced the foundational concepts of Java programming, covering essential topics such as data types, variables, operators, and control structures. You learned the rules for naming variables, the difference between primitive and non-primitive data types, and the various operators used to perform computations and decisions.**
- **The unit also explained how to write, compile, and run simple Java programs, highlighting the role of the main method. Additionally, you explored methods for handling user input using Scanner, BufferedReader, and Console classes, as well as different ways to display output with print, println, and printf.**